apparent that the code accesses shared data. However, with time you learn which library calls these are, and they stick out like sore thumbs. You can then either add appropriate protection and synchronization or use an alternate function that's safe for concurrent access from multiple threads.

There's more to designing multithreaded code for testability than structuring your code to minimize the amount of code that needs to deal with concurrency-related issues and paying attention to the use of non-thread-safe library calls. It's also helpful to bear in mind the same set of questions you ask yourself when reviewing the code, from section 10.2.1. Although these questions aren't directly about testing and testability, if you think about the issues with your "testing hat" on and consider how to test the code, it will affect which design choices you make and will make testing easier.

Now that we've looked at designing code to make testing easier, and potentially modified the code to separate the "concurrent" parts (such as the thread-safe containers or state machine event logic) from the "single-threaded" parts (which may still interact with other threads through the concurrent chunks), let's look at the techniques for testing concurrency-aware code.

10.2.4. Multithreaded testing techniques

So, you've thought through the scenario you wish to test and written a small amount of code that exercises the functions being tested. How do you ensure that any potentially problematic scheduling sequences are exercised in order to flush out the bugs?

Well, there are a few ways of approaching this, starting with brute-force testing, or stress testing.

Brute-Force Testing

The idea behind brute-force testing is to stress the code to see if it breaks. This typically means running the code many times, possibly with many threads running at once. If there's a bug that manifests only when the